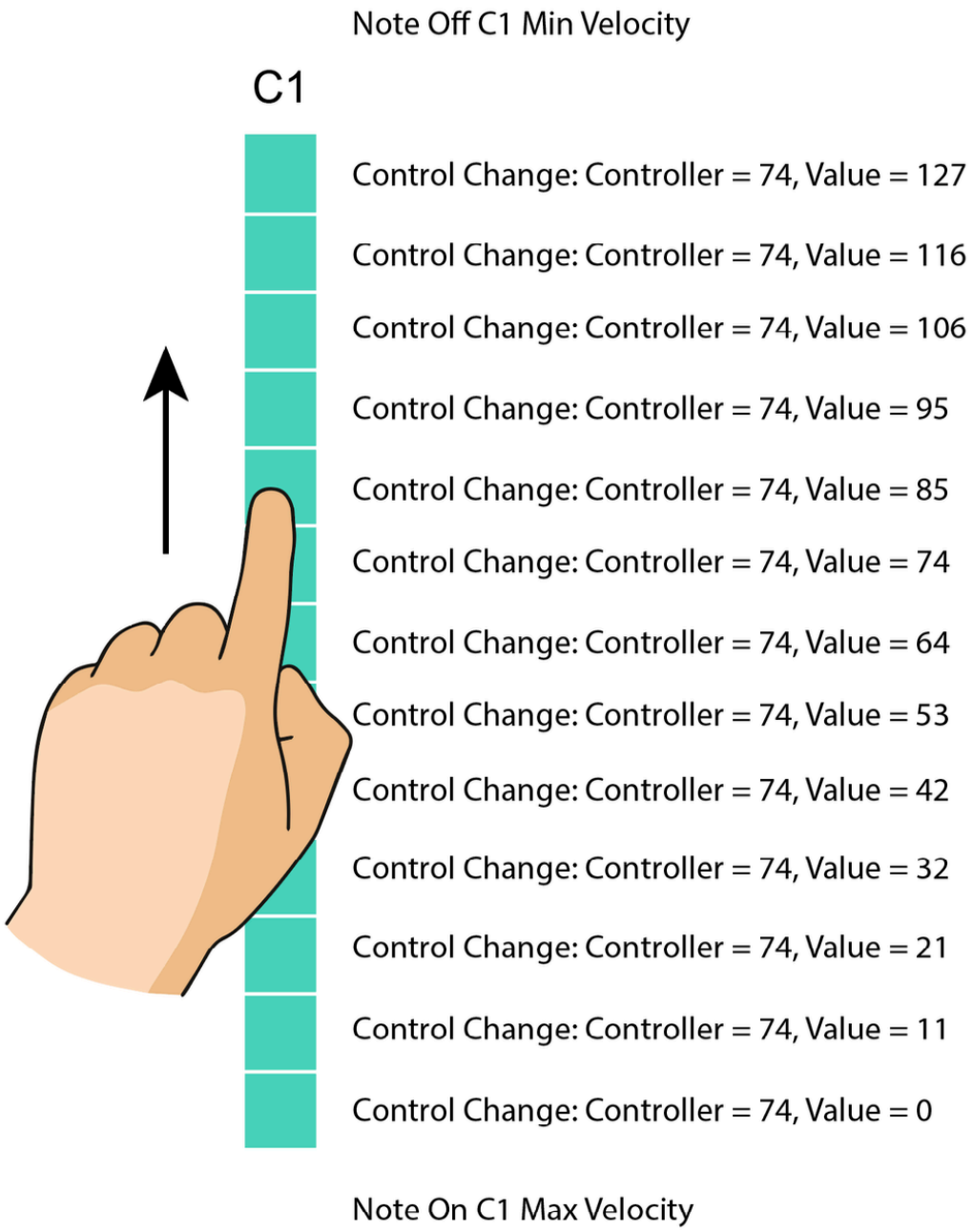
Embedded MIDI Keyboard

A PCB embedded at the bottom of a custom guitar, with capacitive touch electrodes that function like keyboard keys. Touching keys sends MIDI messages over Bluetooth.

Each of the `13` keys is divided into `24` electrodes and functions like a capacitive touch slider, allowing the user to slide their finger up and down on each key to control MIDI "brightness" parameter (`CC74`).



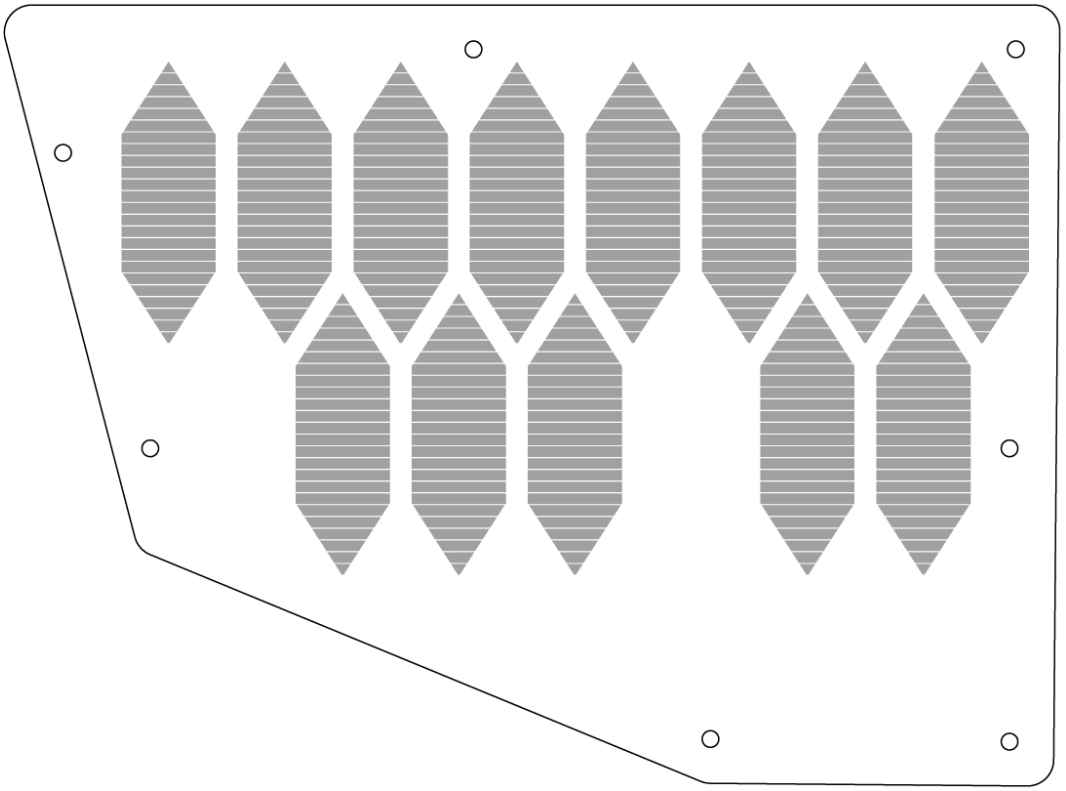
> *Note that the keyboard is mounted "upside down" for easy access when wearing the guitar.*

Overview

- All MIDI messages are sent on channel `1`
- Touching any electrode of a key will send a MIDI `Note On` message with maximum velocity and the note value set according to the key index
- When touch is no longer registered on a key, the controller will send a MIDI `Note Off` message with minimum velocity
- The system is strictly **monophonic**:
  - At most one key may be active at any time

    - At most one electrode position is considered active at any time
- Depending on which key electrode was touched (`0` through `23`), the controller will send a `Control Change` message with controller `74` (or `4AH`, mapped to *brightness* in MIDI specification) and value equal to the index of the key electrode mapped onto `0` through `127` range.

The board outline is provided as an `.svg` export. This includes the shape of the board, hole placement, and the shape of the electrodes:



The capacitive touch electrodes must be placed on one side of the board, and all components on the other.

Scope

The freelancer is responsible for:

- Schematic design
- PCB layout for a custom-shaped board
- Component selection (*suggested* components are listed at the end of this document)
- Manufacturing-ready files (Gerbers, drill, pick-and-place, BOM)

The freelancer is **not** responsible for:

- Firmware development
- Mechanical CAD of the guitar body
- Board outline and mounting hole placement

Requirements

General

- All MIDI messages are sent on channel `1`
- MIDI Control Change `CC74` (`0x4A` , Brightness) is always sent when a key is touched or released
- Control Change value is derived from the closest electrode index (`0`–`23`) mapped linearly to `0`–`127`

Touch behavior

Case 1: Touch a key when no other key is active

- Given my finger is over the `C1` key
- When I touch any electrode within this key
  - Then the controller sends **MIDI Control Change**
    - Channel: `1`
    - Controller: `CC74`
    - Value: `0`–`127` (mapped from electrode index)
  - Then the controller sends **MIDI Note On**
    - Channel: `1`
    - Pitch: `C1`
    - Velocity: `127`

Case 2: Move to another key while holding a key

- Given I am holding the `C1` key
- When I touch any electrode within the `C2` key
  - Then the controller sends **MIDI Note Off**
    - Channel: `1`
    - Pitch: `C1`
    - Velocity: `0`
  - Then the controller sends **MIDI Control Change**
    - Channel: `1`
    - Controller: `CC74`
    - Value: `0`–`127` (mapped from electrode index)
  - Then the controller sends **MIDI Note On**
    - Channel: `1`
    - Pitch: `C2`
    - Velocity: `127`

Case 3: Release the active key

- When I release the active key
  - Then the controller sends **MIDI Control Change**
    - Channel: `1`
    - Controller: `CC74`
    - Value: `0`–`127`
  - Then the controller sends **MIDI Note Off**
    - Channel: `1`
    - Pitch: last active pitch

    - Velocity: `0`

Testing

MIDI Monitor can be used for testing this device once assembled to ensure it adheres to above requirements.

Architecture

Controller

The ESP32-S3-DEVKITC-1-N8 controller was chosen because it can function like a Bluetooth MIDI device, and it's also used for the embedded oscilloscope board in the same project.

The ESP32-S3 shall expose all required GPIO, I²C, UART, boot, and reset signals on the PCB without relying on bodge wires.

Capacitive Touch

The MPR121QR2 capacitive touch controller was selected due to its simplicity, reliability, and native I²C interface. Each MPR121 provides **12 capacitive electrodes**, and multiple devices can coexist on the same I²C bus using its **four selectable I²C addresses (0x5A–0x5D)**.

This project requires **13 keys × 24 electrodes per key = 312 total electrodes**, corresponding to **26 MPR121 devices** (2 per key).

To minimize device count, routing complexity, and I²C bus capacitance, the design shall use a **single TCA9548A I²C multiplexer**. Each TCA9548A downstream channel may host **up to four MPR121 devices**, differentiated by their I²C addresses. With this approach, all 26 MPR121 devices can be supported using **7 of the 8 available TCA9548A channels**, leaving one channel unused for expansion or debugging.

This architecture is preferred over a "one-sensor-per-mux-channel" approach because it:

- Minimizes the number of multiplexers
- Reduces PCB routing congestion
- Keeps I²C timing deterministic
- Simplifies firmware scanning (each sensor is read once per scan cycle)

Each key shall be treated logically as a **1D capacitive slider**, composed of **24 contiguous electrodes**, with position derived in firmware from the combined electrode state.

> *Note: each MPR121 device has an IRQ (interrupt) pin pulled high when touch is registered. The same four MPR121 devices that share a single mux channel also share a single interrupt.*

Layout

- Board layout, hole placement, and electrode copper areas will be provided in DWG/SVG format
- Each electrode shall be routed as a dedicated copper area (no shared pads)
- No electrode shall be stitched together electrically
- Guard rings, ground pours, or shielding may be added only if they do not reduce sensitivity
- Electrode routing shall prioritize uniform length and spacing within each key

Power

This section is a **direct reuse** of a previously validated "Embedded Oscilliscope" design and should not be functionally modified unless required for layout reasons.

A USB-C receptacle is used to program the board, providing power during the programming process. Two TS-1064S-A1B2-D4 switches (boot and reset) are used for manual programming. A DMN3135LVT-7 MOSTFET enables automatic programming from Arduino IDE.

The circuitry from Adafruit LiPo backpack is integrated directly onto the board to provide power from a LiPo battery after the device is programmed. This includes S2B-PH-SM4-TB **battery connector** and a **sliding power switch**. If the battery is connected while power is provided to the USB-C connector, this also charges the battery.

The power switch (connected to LiPo Battery Backpack circuit) is **external** for this project - but S2B-PH-SM4-TB connector will be needed on the board to connect this external switch in a modular fashion. This is exactly the same connector as the one used for the battery.

The circuitry from Adafruit LM3671 Buck Converter is integrated directly onto the board to convert the battery voltage to `3.3V` required for the controller and all other components.

Components

Component	Description
ESP32-WROOM-32E-N4	Controller
MCP73831T-2ACI/OT	Battery Charger
150080SS75000	Red LED
150080GS75000	Green LED
RE0805FRE071KL	1K Resistor
RC0805FR-072K49L	2.5K Resistor
0402WGF5101TCE	5.1K Resistor
AF0805FR-0710KL	10K Resistor
RT0805FRE0775KL	75K Resistor
RC0805FR-07100KL	100K Resistor
RTT01513JTH	51K Resistor
S2B-PH-SM4-TB	LiPo Battery Connector, Power Switch Connector
CC0805KKX5R5BB106	10uF Capacitor
CC0805MKX5R5BB226	22uF Capacitor
CL03A104KP3NNNC	100nF Capacitor
LM3671MFX-3.3/NOPB	3.3V DC-DC Buck Converter
NRH2412T2R2MNGH	2.2uH Inductor

TS-1064S-A1B2-D4	Boot and Reset Switches
USB-C31-S-RA-CS2-SMT-BK-T/R	USB-C Receptacle
DMN3135LVT-7	ESP32 Auto-program MOSFET
CH340C	USB to Serial Converter
MPR121QR2	Capacitive Touch Sensor
TCA9548APWR	I2C Multiplexer (Bus Switch)